Software Architecture and Design Specification

Project: Event Management System

Version: 1.0

Team: Team 7

Date: 04-10-2026

Status: Draft / Initial SAD

Field	Details
Author	Aryan Mittal · PES2UG24CS089
Author	Ashmita Chaki · PES2UG24CS090
Author	Ashwath Rohan · PES2UG24CS091
Author	Atharv Kulkarni · PES2UG24CS092
Baseline	Event Management System SRS v1.0; Software Test Plan v1.0

Revision History

Version	Date	Author	Change Summary	Approval
1.0	04-10-2026	Aryan Mittal Ashmita Chaki Ashwath Rohan Atharv Kulkarni	Initial SAD for Event Management System based on SRS v1.0, STP v1.0 and supplied SAD template.	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Project Team	Aryan Mittal	aryanmittal2107@gmail.com	04-10-2026
Project Team	Ashmita Chaki	chakiashmita@gmail.com	04-10-2026
Project Team	Ashwath Rohan	ashwathrohan16@gmail.com	04-10-2026
Project Team	Atharv Kulkarni	starcak06@gmail.com	04-10-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Event Management System (EMS). It translates the baseline functional, non-functional, and security requirements from the SRS into a consistent software architecture, component structure, interaction design, API design, security architecture, and operational design.

1.2 Scope

The SAD covers the software architecture for event creation and management, event publication and discovery, participant registration, registration monitoring, participant/organizer/administrator functions, reporting, validation, audit logging, authentication, authorization, and secure client-server communication. Physical venue management, external payment settlement, institution-wide identity infrastructure, and unapproved third-party services remain outside the baseline scope.

1.3 Audience

Developers, QA/test engineers, security reviewers, course instructors/evaluators, project team members, and future maintainers of the EMS.

1.4 Definitions

EMS — Event Management System; Participant — user who views and registers for events; Organizer — authorized user who creates and manages events; Administrator — authorized system-level user; API — Application Programming Interface; RBAC — Role-Based Access Control; RTM — Requirements Traceability Matrix; SAD — Software Architecture and Design Specification; STP — Software Test Plan.

2. Document Overview

2.1 How to use this document

Section 3 defines the architecture, stakeholders, components, architecture pattern, technology stack, risks, requirement traceability, and security architecture. Section 4 describes the detailed design, sequence flows, API interfaces, error handling/logging/monitoring, UX considerations, and open issues. Section 5 provides the glossary, source references, design decisions (ADRs), and design tools.

2.2 Related Documents

- Event Management System Software Requirements Specification (SRS), Version 1.0, dated 03-09-2026.
- Event Management System Software Test Plan (STP), Version 1.0, dated 16-09-2026.
- Requirements Traceability Matrix contained in the EMS SRS v1.0.

3. Architecture

3.1 Goals & Constraints

Architecture goals:

- Provide clear separation of presentation, application/business logic, administrative/data-management functionality, and persistence.
- Centralize event information and registration workflows for Participants, Organizers, and Administrators.
- Enforce authenticated, server-side role and ownership checks for protected operations.
- Support the SRS performance target of a 90th-percentile user-facing API response time of 3 seconds or less under the project test load.
- Support the SRS availability target of at least 99.0% monthly availability excluding planned maintenance.
- Provide responsive and accessible browser-based interaction and auditable security-relevant operations.

Architecture constraints:

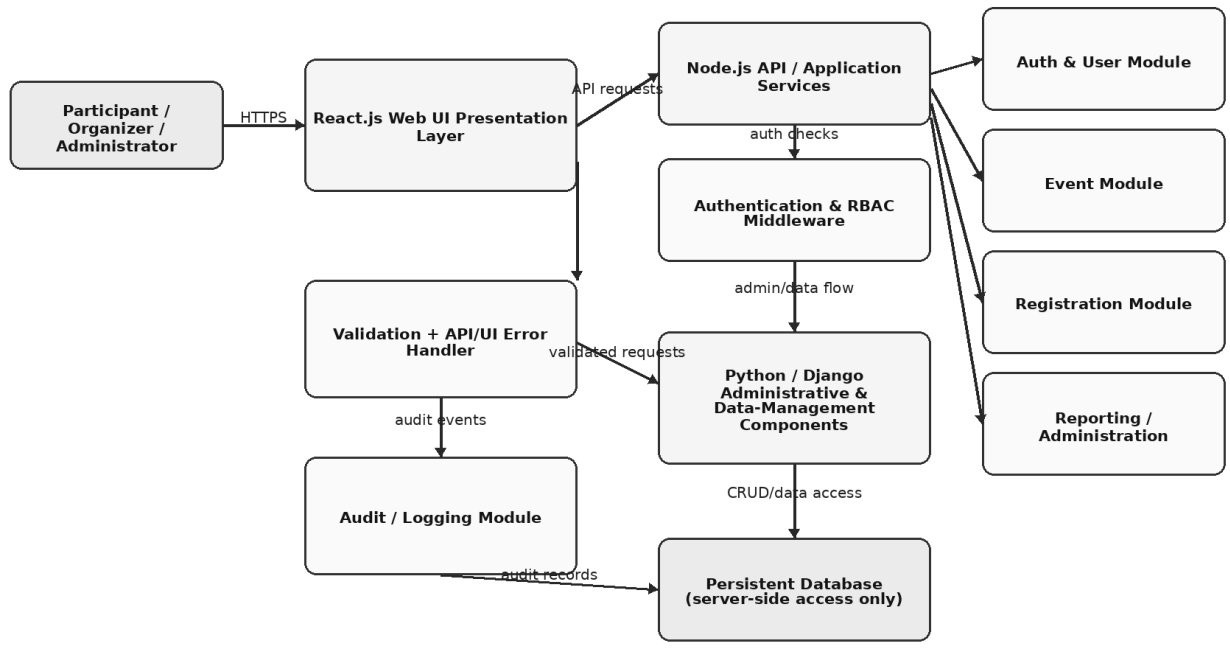
- Approved implementation stack: React.js and Node.js, plus Python/Django for the defined Python-based administrative/data-management components.
- The browser is the primary client environment; no dedicated hardware interface is required.
- The database is accessed through server-side application components rather than directly from the client.
- Deployed client-server communication shall use HTTPS/TLS.
- External payment, SMS, third-party identity infrastructure, and other third-party services are not baseline dependencies unless separately approved and documented.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Participant	Simple event discovery, correct availability, successful registration, privacy, clear errors, responsive UI.
Organizer	Event creation/editing/publishing/cancellation, ownership restrictions, participant lists, reliable updates.
Administrator	Authorized user/event oversight, administrative reporting, auditability and security.
Project Team / Developers	Modularity, maintainability, clear interfaces, consistent validation and error handling.
QA / Test Engineers	Requirement-to-design traceability, testable interfaces, measurable performance/security criteria.
Course Coordinator / Evaluator	Conformance to SRS, acceptance criteria, traceability, and demonstrable architecture.

3.3 Component (UML) Diagram

Figure 1. UML Component Diagram — Event Management System



Boundary: browser communicates with server over HTTPS in deployment; protected operations are authorized server-side.

The component design uses React.js as the presentation layer, Node.js as the primary application/API layer, Python/Django for the defined administrative/data-management functionality, and a persistent database accessed through the server side. Validation, error handling, authentication/RBAC, and audit logging are treated as cross-cutting responsibilities.

3.4 Component Descriptions

Component	Responsibility	Key Requirements / Interfaces
React.js Web UI	Event discovery, event details, registration, organizer event management, administrator UI, forms and user feedback.	EMS-F-006 to EMS-F-009, EMS-F-012; EMS-NF-005; communicates with server APIs.
Node.js API / Application Services	Exposes application endpoints and business logic for authentication, events, registrations, reporting and related workflows.	EMS-F-001 to EMS-F-018; EMS-NF-001; server-side business logic.
Authentication & RBAC Middleware	Authenticates requests and enforces role/ownership checks before protected operations.	EMS-F-002, EMS-F-013, EMS-F-014; EMS-NF-003; EMS-SR-001, EMS-SR-002.
Auth & User Module	Account registration, credential handling, login/session or token processing, role identification.	EMS-F-001, EMS-F-002; EMS-SR-005.
Event Module	Create, edit, publish, cancel, list and retrieve event details; maintains event state and capacity.	EMS-F-003 to EMS-F-008, EMS-F-017.
Registration Module	Creates registrations, prevents duplicates, checks capacity/event state, and supports registration records.	EMS-F-009 to EMS-F-012.
Organizer Module	Provides authorized participant-list management for organizer-owned events.	EMS-F-013; EMS-SR-002.
Administration Module	Provides authorized system-level user/event oversight.	EMS-F-014.
Reporting Module	Produces basic event and registration reports for authorized users.	EMS-F-015.
Validation + API/UI Error Handler	Validates inputs and produces clear failure responses without partial/duplicate records.	EMS-F-017, EMS-F-018; EMS-SR-004.
Audit / Logging Module	Records important event, registration, authentication and administrative actions with timestamp and actor information.	EMS-F-016; EMS-NF-004.
Python / Django Components	Implements the defined Python-based administrative/data-management functionality and its integration boundary.	SRS 2.1, 2.4; integration tested with Node/API and database.
Persistent Database	Stores user, event, registration, and audit-related records through server-side components.	Supports functional persistence and reporting; direct client access is prohibited by architecture.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is selected for the EMS, with modular service boundaries inside the application layer. This preserves the SRS separation between the React.js frontend, Node.js application/API services, Python/Django components, and persistent database. The approach also supports server-side authorization, validation, audit logging, and independent testing of modules.

A full microservices architecture is not selected for the baseline design because the SRS defines a single web application with Node.js services and specified Django functionality rather than independently deployable services. Keeping the modules within a layered application reduces integration and deployment complexity while retaining clear boundaries that can be evolved later if required.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology / Design	Notes
Presentation	React.js	Responsive browser UI for desktop/mobile viewport sizes.
Application / API	Node.js	Server-side endpoints and business logic.
Python-based modules	Python / Django	Defined administrative/data-management functionality.
Persistence	Persistent database (DBMS not specified in SRS)	Accessed through server-side components; schema design follows users, events, registrations and audit needs.
Communication	HTTPS/TLS in deployment	Authenticated session or token information is carried by API requests as defined by implementation.
Testing / operational tooling	Postman, browser developer tools, applicable unit-test framework, JMeter or equivalent	Specified in the STP for API, UI, unit and performance testing.

3.7 Risks & Mitigations

Risk	Mitigation / Architectural Response
Frontend/API integration instability	Keep API contracts explicit; run integration testing early between React.js, Node.js, Django components, database and authentication middleware.
Unauthorized role or ownership access	Enforce authentication and role/ownership checks on the server side; do not rely on client-side hiding of controls.
Invalid or malicious input	Centralize validation/sanitization before processing or persistence and return clear errors.
Duplicate/capacity registration errors	Registration service performs event-state, capacity and duplicate checks before creating a record.
Audit gaps	Route defined auditable actions through the audit/logging module and store timestamp + actor information.
Performance below target	Measure API response time under representative load and keep persistence/business operations on the server side.
Service/API downtime	Use repeatable test/deployment environments and verify startup/recovery and the 99.0% availability target.

3.8 Traceability to Requirements

The following design traceability preserves the SRS baseline. Requirement IDs and test IDs are retained exactly; design module names correspond to the SRS RTM. Status remains N (Not Executed) because the source RTM is a baseline design/test traceability matrix, not an execution result.

Req ID	Requirement Short	Design Module	Test Case
EMS-F-001	User registration	Auth & User Module	TC-REG-01
EMS-F-002	User authentication	Auth & User Module	TC-AUTH-01
EMS-F-003	Create event	Event Module	TC-EVT-01
EMS-F-004	Edit event	Event Module	TC-EVT-02
EMS-F-005	Publish/cancel event	Event Module	TC-EVT-03
EMS-F-006	Event listing	Event Discovery UI	TC-VIEW-01
EMS-F-007	Search/filter events	Event Discovery UI	TC-SEARCH-01
EMS-F-008	Event details / availability	Event Discovery UI	TC-VIEW-02
EMS-F-009	Participant registration	Registration Module	TC-REG-02
EMS-F-010	Prevent duplicate registration	Registration Module	TC-REG-03
EMS-F-011	Capacity control	Registration Module	TC-REG-04
EMS-F-012	View personal registrations	Participant Module	TC-REG-05
EMS-F-013	Organizer participant management	Organizer Module	TC-ORG-01
EMS-F-014	Administrator management	Administration Module	TC-ADM-01
EMS-F-015	Basic reports	Reporting Module	TC-REP-01
EMS-F-016	Audit logging	Audit/Logging Module	TC-AUD-01
EMS-F-017	Input validation	Validation Module	TC-VAL-01
EMS-F-018	Error handling	API/UI Error Handler	TC-ERR-01
EMS-NF-001	API response target	API Services	TC-PERF-01
EMS-NF-002	Availability target	Deployment / Services	TC-OPS-01
EMS-NF-003	Authorization enforcement	Auth Middleware / API	TC-SEC-01
EMS-NF-004	Timestamped audit/security logs	Audit/Logging Module	TC-LOG-01
EMS-NF-005	Responsive/accessibility	React UI	TC-UX-01
EMS-SR-001	Authenticated protected access	Auth Middleware / API	TC-SEC-01
EMS-SR-002	Role and ownership checks	Auth Middleware / API	TC-SEC-02
EMS-SR-003	HTTPS/TLS	Deployment / API	TC-SEC-03
EMS-SR-004	Input validation/sanitization	Validation Module	TC-SEC-04
EMS-SR-005	Password hashing	Auth & User Module	TC-SEC-05

3.9 Security Architecture

The security architecture follows the SRS security objectives: authenticated, role-based protection of user and event-management data; validation of requests; timestamped security-relevant actions; and protected client-server communication.

STRIDE Concern	EMS Design Response	Related Requirements
Spoofing	Authentication middleware validates protected requests and identifies the assigned user role.	EMS-F-002; EMS-NF-003; EMS-SR-001
Tampering	Server-side authorization/ownership checks and input validation protect event/registration modifications.	EMS-F-004, EMS-F-005, EMS-SR-002, EMS-SR-004
Repudiation	Audit/logging module records important actions with timestamp and actor information.	EMS-F-016; EMS-NF-004
Information Disclosure	HTTPS/TLS protects deployed client-server communication; server-side authorization limits	EMS-SR-001, EMS-SR-003, EMS-SR-005

	record access; passwords are stored as hashes rather than plaintext.	
Denial of Service	Validation, bounded business operations and performance testing reduce failure propagation; deployment availability is measured against the SRS target.	EMS-NF-001, EMS-NF-002, EMS-F-018
Elevation of Privilege	RBAC and ownership checks are enforced on the server rather than only in the React UI.	EMS-NF-003; EMS-SR-002; EMS-F-013, EMS-F-014

Note: Exact cryptographic algorithms, session/token technology, rate-limit values, and database vendor are implementation choices not specified by the SRS; they must be selected and documented during implementation without weakening the stated security requirements.

4. Design

4.1 Design Overview

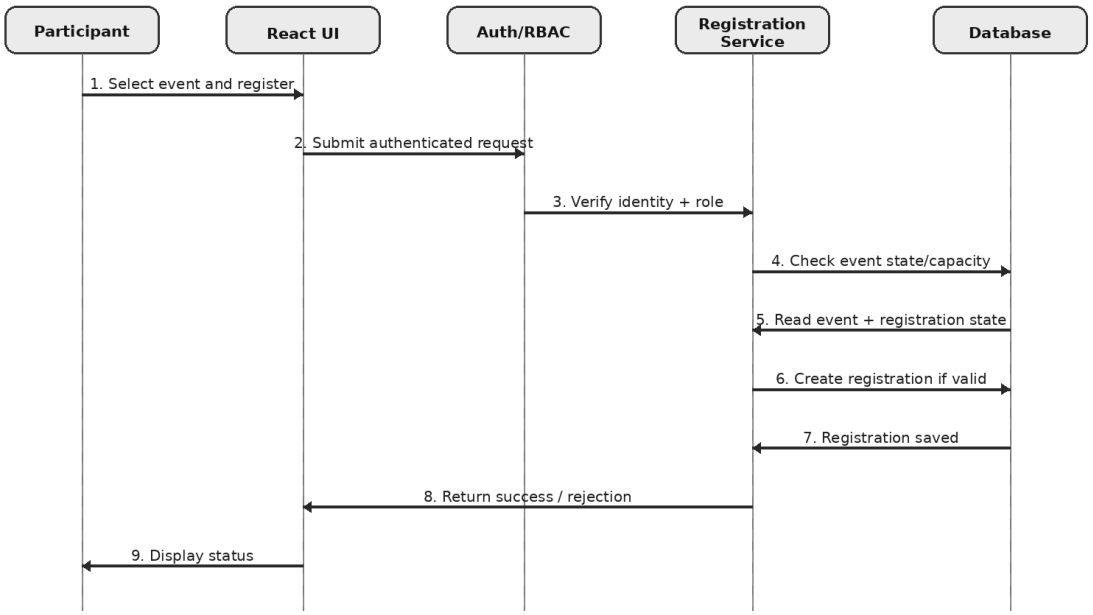
The EMS design follows a request-response web architecture. React.js handles presentation and user interaction. Node.js receives API requests, applies authentication/authorization, validates inputs, invokes the relevant business module, and accesses persistence through server-side components. Python/Django provides the specified administrative/data-management functionality. Audit/logging records important actions. Errors are converted into clear UI/API feedback without creating partial or duplicate records.

- Participant flow: discover event → view details/availability → register → validate identity/event state/capacity/duplicate status → persist registration → show result.
- Organizer flow: authenticate → create/edit event → validate date/time/capacity → persist → optionally publish/cancel → reflect status in event listings.
- Administrator flow: authenticate with administrator role → access authorized user/event oversight → generate basic reports → actions are auditable.

4.2 UML Sequence Diagrams

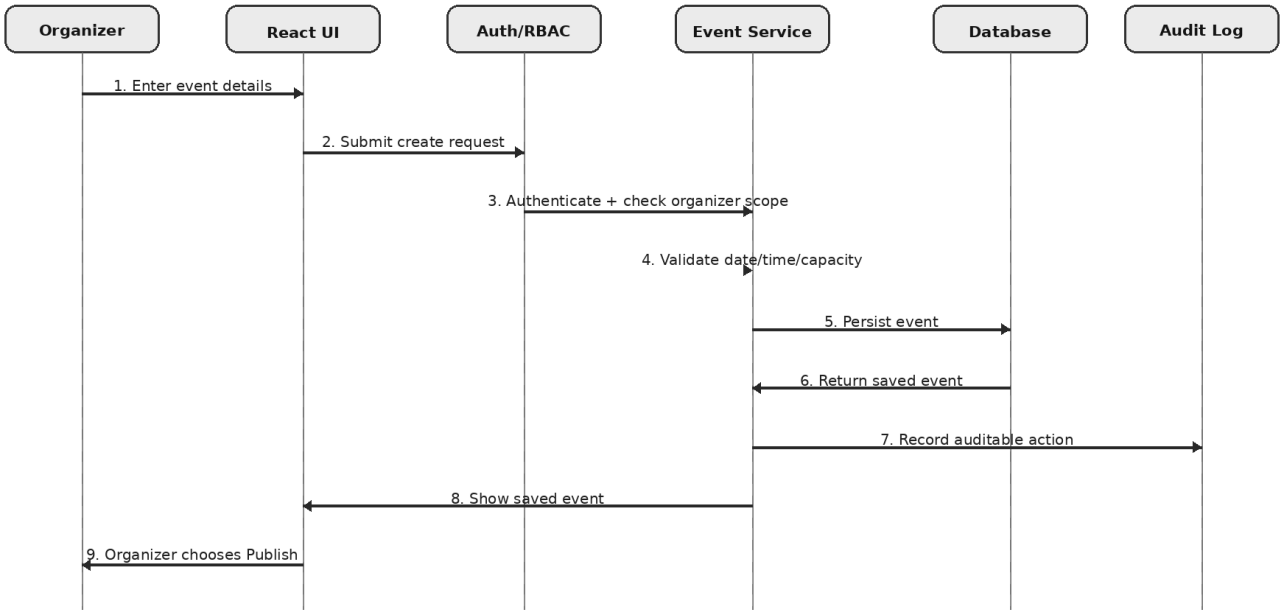
The following two sequence diagrams cover the primary registration flow and the organizer event-management flow required by the SAD template.

Figure 2. UML Sequence Diagram — Participant Registration



Validation prevents duplicates and blocks registration when capacity is zero; failure states return clear messages without partial records.

Figure 3. UML Sequence Diagram — Organizer Creates and Publishes Event



Publish/cancel actions use the same authorization boundary and are reflected in event listings.

4.3 API Design

The SRS defines that React.js communicates with server-side APIs for authentication, events, registrations and reports, but it does not prescribe concrete endpoint names or payload schemas. The following REST-style interfaces are therefore proposed design interfaces consistent with the SRS; they may be refined during implementation while preserving requirement intent.

Component / Endpoint	Method	Request (illustrative)	Response / Errors
Auth API — /api/auth/register	POST	name, email, password, role (where permitted)	201 account created; 400 invalid/missing fields; 409 duplicate account.
Auth API — /api/auth/login	POST	email, password	200 authenticated session/token + role; 401 invalid credentials.
Event API — /api/events	POST	title, description, date, time, venue, capacity	201 event created; 400 validation error; 401/403 authorization error.
Event API — /api/events/{eventId}	PATCH	Fields to update for an authorized event	200 updated event; 403 ownership/permission error; 404 event not found; 400 invalid data.
Registration API — /api/events/{eventId}/registrations	POST	Authenticated participant context	201 registration created; 409 duplicate/full/closed; 401 unauthenticated; 400 invalid request.
Registration API — /api/registrations/me	GET	Authenticated participant context	200 active/past registration records; 401 unauthenticated.
Reporting API — /api/reports/registrations	GET	Authorized scope/filter parameters	200 report data; 403 unauthorized scope; 400 invalid filter.

Interface rule: protected endpoints must enforce authorization on the server. HTTP responses should use consistent status semantics and user-facing messages should avoid exposing sensitive implementation details.

4.4 Error Handling, Logging & Monitoring

- Validation errors: reject invalid/missing values with field-level or operation-level feedback; invalid date/time/capacity values must not be persisted.
- Authorization errors: reject unauthenticated or insufficiently authorized requests; do not expose protected records.
- Conflict errors: prevent duplicate registrations and registration when capacity is zero; no duplicate record is created.
- Server errors: return a generic, understandable error message to the UI and avoid creating partial records.
- Audit logs: record defined authentication, event, registration and administrative actions with timestamp and actor information.
- Security logging: do not store passwords or sensitive credentials in plaintext logs.
- Monitoring: track user-facing API response performance, availability/service health, authentication/authorization failures, and important error rates so the SRS targets can be evaluated.

4.5 UX Design

The React.js interface shall provide clear navigation for event discovery, event management, registration and administration. Forms shall use meaningful labels, validation messages, success/error feedback, and confirmation prompts for destructive actions such as cancellation or deletion. The UI shall remain understandable on common desktop and mobile viewport sizes and provide keyboard-accessible controls where applicable.

UX Area	Design Response
Event discovery	Event list with title, date, venue and current status; search/filter support.
Event details	Show title, description, date, time, venue, capacity/availability and registration state.
Registration	Clear register action, success confirmation, and explicit full/closed/duplicate feedback.
Organizer management	Event creation/editing forms with validation and publish/cancel confirmation.
Administration	Role-restricted administrative navigation and clear reporting/oversight views.
Accessibility	Meaningful labels, readable feedback, responsive layouts, and keyboard-accessible controls where applicable.

4.6 Open Issues & Next Steps

- Select and document the concrete database management system and physical schema; the SRS only specifies a persistent database.
- Finalize session/token mechanism and exact password-hashing configuration while meeting EMS-SR-005.

- Finalize concrete API endpoint naming, payload schemas, and error-code conventions before implementation freeze.
- Define the deployment topology for achieving and measuring the 99.0% availability target.
- Define the exact auditable action set and log retention policy.
- Complete UML diagrams for implementation-level classes/data model if required by the course deliverables.
- Execute the STP test suite and update requirement/test status from N to P or A based on actual execution evidence.

5. Appendices

5.1 Glossary

Term	Meaning
EMS	Event Management System.
Participant	General user who views events, registers, and reviews personal registrations.
Organizer	Authorized user who creates/manages events and their participant information.
Administrator	Authorized user responsible for system-level administration and oversight.
API	Application Programming Interface used for communication between software components.
RBAC	Role-Based Access Control.
RTM	Requirements Traceability Matrix connecting requirements to design modules and test cases.
SAD	Software Architecture and Design Specification.
SRS	Software Requirements Specification.
STP	Software Test Plan.

5.2 References

- Event Management System SRS v1.0 (03-09-2026).
- Event Management System STP v1.0 (16-09-2026).
- Supplied Software Architecture and Design Specification template (ATM example).

5.3 Architecture Decision Records (ADRs)

ADR	Decision	Rationale	Status
ADR-001	Use layered architecture with modular application services.	Matches the SRS web stack and separates UI, application logic, Django functionality and persistence without unnecessary microservice complexity.	Accepted for v1.0
ADR-002	Use React.js for presentation and Node.js for primary application/API services.	Directly follows the SRS product perspective and external interface requirements.	Accepted for v1.0
ADR-003	Keep Python/Django as a defined administrative/data-management component.	The SRS explicitly assigns Python/Django to defined Python-based administrative/data-management functionality.	Accepted for v1.0
ADR-004	Enforce authorization server-side.	The SRS explicitly prohibits relying only on client-side hiding of protected controls and requires role/ownership checks.	Accepted for v1.0
ADR-005	Access the database only through server-side components.	The SRS defines database communication through the server-side application layer, reducing direct client access and centralizing validation/authorization.	Accepted for v1.0

5.4 Tools

- Microsoft Word / python-docx for document preparation.
- UML-style component and sequence diagrams prepared for this SAD.
- Postman or equivalent for API testing, browser developer tools, applicable unit-test framework, and JMeter or equivalent, as listed in the STP.